

WPROWADZENIE DO PROGRAMOWANIA W PHP

SPIS TREŚCI

Spis treści	1
Cel zajęć.....	1
Uwaga	1
Pobranie i uruchomienie PHP	2
Instalacja Node.js i LESS	3
Pobranie i konfiguracja frameworka	4
XDEBUG.....	6
(Alternatywnie) VS Code	7
Zatrzymanie w pułapce	9
Omówienie frameworka	9
Rozszerzenie aplikacji	10
Commit projektu do GIT.....	14
Podsumowanie.....	14

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- konfiguracji środowiska do programowania w PHP;
- połączenia z bazą danych;
- konfiguracji narzędzi do debugowania;
- analizy istniejących systemów webowych bez dokumentacji;
- uzupełniania istniejących systemów webowych o nowe funkcjonalności.

W praktycznym wymiarze uczestnicy zapoznają się z istniejącym naiwnym frameworkiem do tworzenia systemów webowych i uzupełnią go o nową funkcjonalność.

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do **Plik** -> **Informacje** -> **Właściwości** -> **Właściwości zaawansowane** -> **Niestandardowe** i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub **Ctrl+A** -> **F9**.

POBRANIE I URUCHOMIENIE PHP

Zaloguj się do systemu Windows / pulpitu zdalnego rdp.wi.zut.edu.pl:

- spoza sieci ZUT potrzebny VPN: <https://uci.zut.edu.pl/uslugi-uci/vpn.html>;
- nazwa użytkownika: WIAD\ab12345
- komputer: rdp.wi.zut.edu.pl

Odwiedź stronę <https://windows.php.net/download/>. Pobierz PHP 8.2.* x64 NTS.

Wypakuj pobrane repozytorium do I:\php.

Otwórz panel sterowania. W polu wyszukiwania wpisz path. Wybierz edycję zmiennych środowiskowych użytkownika. Znajdź zmienną Path i kliknij edycję. Dodaj ścieżkę I:\php.

UWAGA! W toku studiów być może do Path zostały dodane inne wersje PHP lub XAMPP. Za kilka kroków sprawdzana będzie wersja PHP – upewnij się, że faktycznie wiesz z której instancji PHP korzystasz przez resztę tego laboratorium 😊.

Skopiuj plik I:\php\php.ini-development jako php.ini, po czym edytuj jego zawartość – odkomentuj poniższe ustawienia:

```
extension_dir = "ext"
...
extension=curl
extension=gd
extension=intl
extension=mbstring
extension=openssl
extension=pdo_sqlite
```

Otwórz terminal PowerShell i wejdź do katalogu laboratoriów.

Wykonaj komendę

```
php -i | Select-String -Pattern '(PHP Version)|(extension_dir)|(OpenSSL support)|(PDO drivers)|(GD Support)|intl|(cURL support)|multibyte'
```

Oczekiwany wynik:

```
PS C:\Users\artur\workspace\AI2-lab\labA> php -i | Select-String -Pattern '(PHP Version)|(extension_dir)|(OpenSSL support)|(PDO drivers)|(GD Support)|intl|(cURL support)|multibyte'
```

```
PHP Version => 8.2.10
Zend Multibyte Support => provided by mbstring
PHP Version => 8.2.10
extension_dir => ext => ext
zend.multibyte => Off => Off
cURL support => enabled
GD Support => enabled
intl
intl.default_locale => no value => no value
intl.error_level => 0 => 0
intl.use_exceptions => Off => Off
Multibyte Support => enabled
Multibyte string engine => libmbfl
Multibyte (japanese) regex support => enabled
Multibyte regex (oniguruma) version => 6.9.8
OpenSSL support => enabled
PDO drivers => sqlite
OpenSSL support => enabled
```

Zastąp poniższy obrazek swoim zrzutem ekranu:

```

Windows PowerShell
libxml
mbstring
mysqli
mysqlnd
openssl
pcre
PDO
pdo_mysql
pdo_sqlite
Phar
random
readline
Reflection
session
SimpleXML
SPL
sqlite3
standard
tokenizer
xml
xmlreader
xmlwriter
zlib

[Zend Modules]

PS E:\Aplikacje-Internetowe-1\lab-f> php -i | Select-String -Pattern '(PHP Version)|(extension_dir)|(OpenSSL support)|(PDO drivers)|(GD Support)|(intl)|(cURL support)|multibyte'

PHP Version => 8.3.12
Zend Multibyte Support => provided by mbstring
PHP Version => 8.3.12
zend.multibyte => Off => Off
cURL support => enabled
Multibyte decoding support using mbstring => enabled
GD Support => enabled
intl
intl.default_locale => no value => no value
intl.error_level => 0 => 0
intl.use_exceptions => Off => Off
Multibyte Support => enabled
Multibyte string engine => libmbfl
Multibyte (japanese) regex support => enabled
Multibyte regex (oniguruma) version => 6.9.8
OpenSSL support => enabled
PDO drivers => mysql, sqlite
OpenSSL support => enabled

PS E:\Aplikacje-Internetowe-1\lab-f>

```

Punkty:

0

1

INSTALACJA NODE.JS I LESS

Ta sekcja częściowo mogła zostać zrealizowana podczas wcześniejszych laboratoriów. Upewnij się, że zainstalowana jest w miarę bieżąca wersja Node.js.

Wejdź na stronę <https://nodejs.org/en/download/current>. Pobierz wersję Current -> Windows Binary (.zip) -> 64-bit. Rozpakuj archiwum do I:\node. Dodaj ten folder do zmiennej środowiskowej Path użytkownika.

Uruchom nowe okno wiersza poleceń. Wykonaj polecenie `npm -v`. Powinno zadziałać – udało się zainstalować NODE i NPM lokalnie dla użytkownika.

Następnie zainstalujemy Less. Wykonaj polecenie:

```
npm install -g less less-plugin-clean-css
```

Sprawdź poprawność instalacji:

```
lessc -v
```

Wstaw zrzut ekranu wyniku działania poleceń `npm -v` i `lessc -v`

```
PS E:\Aplikacje-Internetowe-1\lab-f> npm -v
10.9.2
C:\Users\Combo>lessc -v
lessc 4.2.1 (Less Compiler) [JavaScript]
```

Punkty:	0	1
---------	---	---

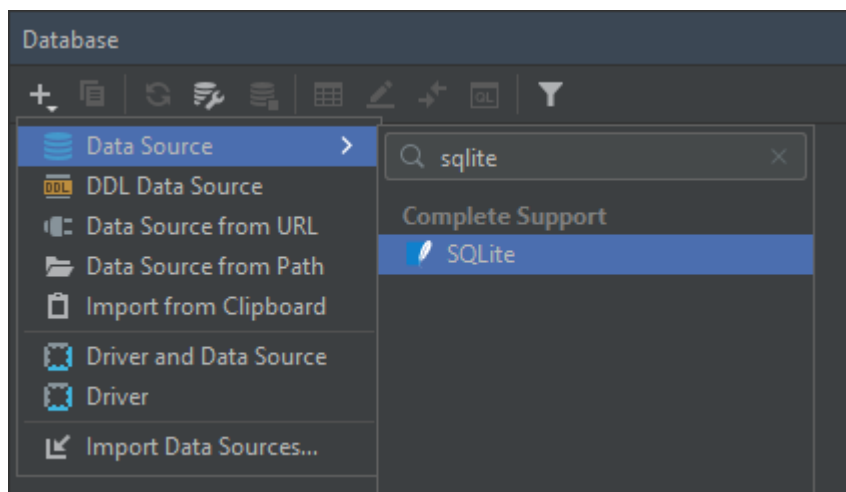
POBRANIE I KONFIGURACJA FRAMEWORKA

Sklonuj naiwny framework z Githuba: <https://github.com/IdeaSpotPL/custom-php-framework>.

Wejdź do pobranego katalogu i skompiluj style:

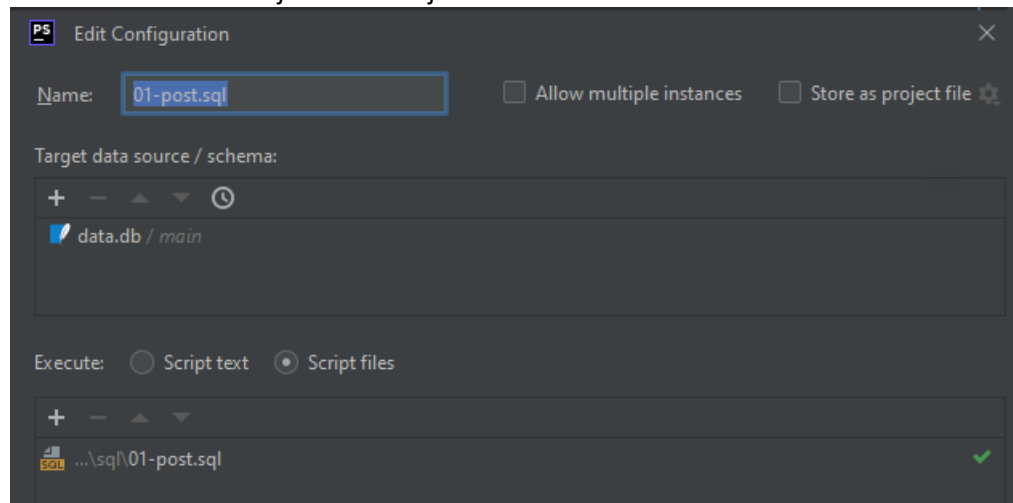
```
lessc public\assets\src\less\style.less public\assets\dist\style.min.css --clean-css
--source-map
```

Otwórz projekt w PhpStorm. Rozwiń panel Database z prawej strony. Dodaj nowe źródło danych SQLite:



W sekcji File kliknij przycisk `New +` i utwórz plik `data.db` w katalogu głównym frameworka. Pobierz brakujące sterowniki, jeśli PhpStorm o to poprosi.

Kliknij prawym przyciskiem w drzewie projektu na `sql/01-post.sql` i wybierz z menu pozycję `Run (...)`. Dostosuj konfigurację jak poniżej i kliknij `Run`.

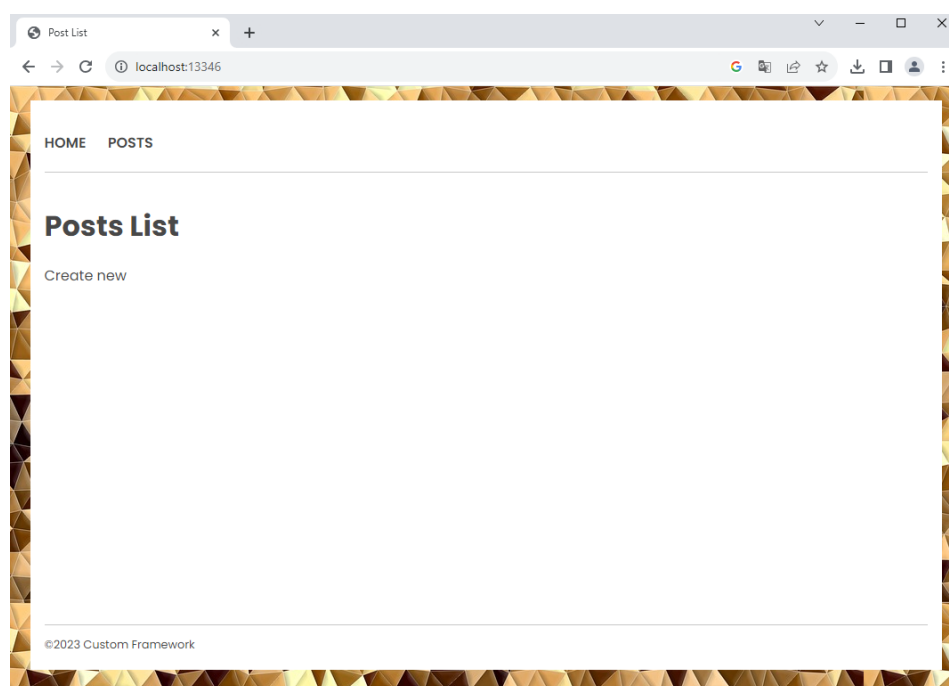


Skopiuj plik `config\config.dist.php` na `config\config.php` i dostosuj ustawienia bazy danych.

Terminalem wejdź do katalogu `public` we frameworku i uruchom wbudowany serwer PHP:

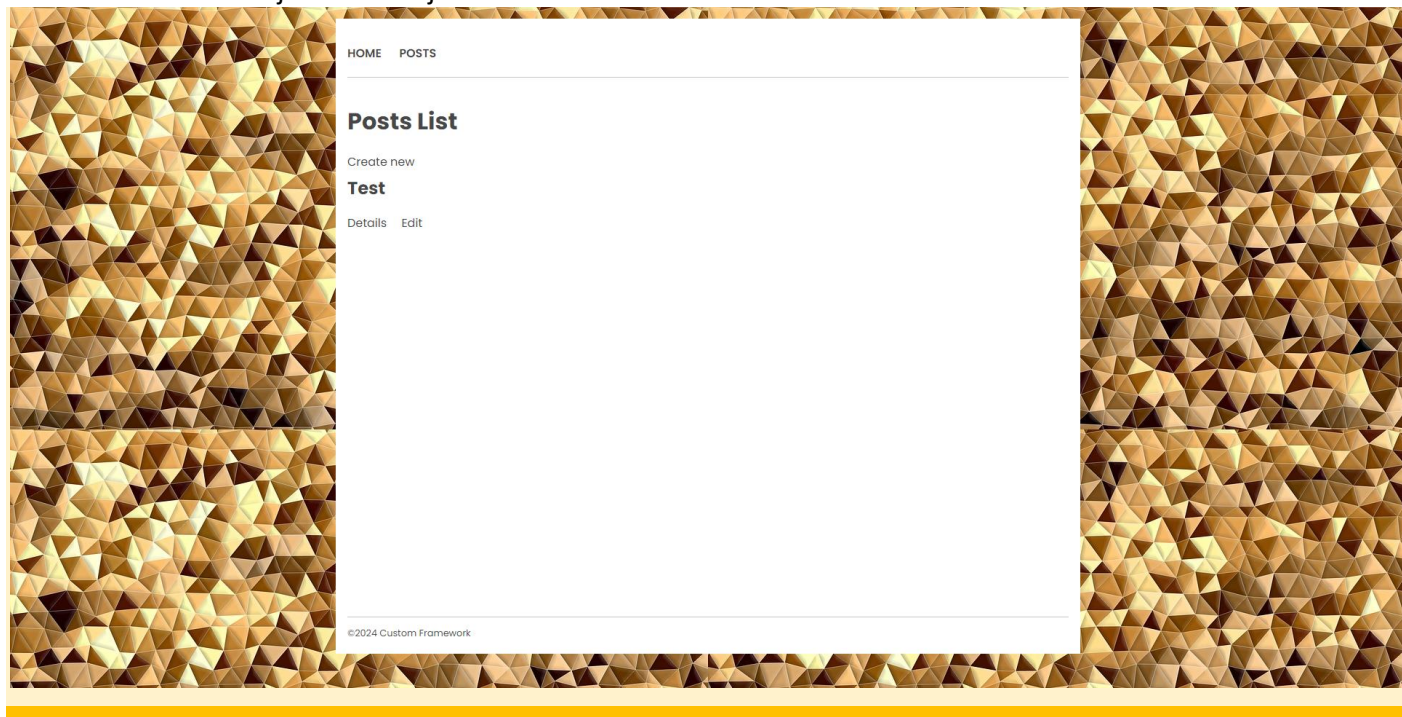
```
php -S localhost:50962
```

Wejdź przeglądarką na powyższy adres. Framework powinien się wyświetlić:



Utwórz nowy wpis poprzez kliknięcie na `Create new`, wypełnienie przykładowym wpisem z Twoimi danymi i wysyłkę formularza.

Wstaw zrzut ekranu listy postów po dodaniu nowego wpisu z Twoimi danymi.



Punkty:	0	1
---------	---	---

XDEBUG

Wejdź na stronę <https://xdebug.org/wizard>. Postępuj zgodnie z instrukcjami, tj:

- skopiuj zawartość strony `http://localhost:<port>/?action=info` do okienka kreatora
- pobierz wskazany plik DLL do odpowiedniego katalogu
- zmodyfikuj zawartość `php.ini` ze wskazanej ścieżki
- zrestartuj serwer WWW.

Przykładowe ustawienia do wpisania do `php.ini`:

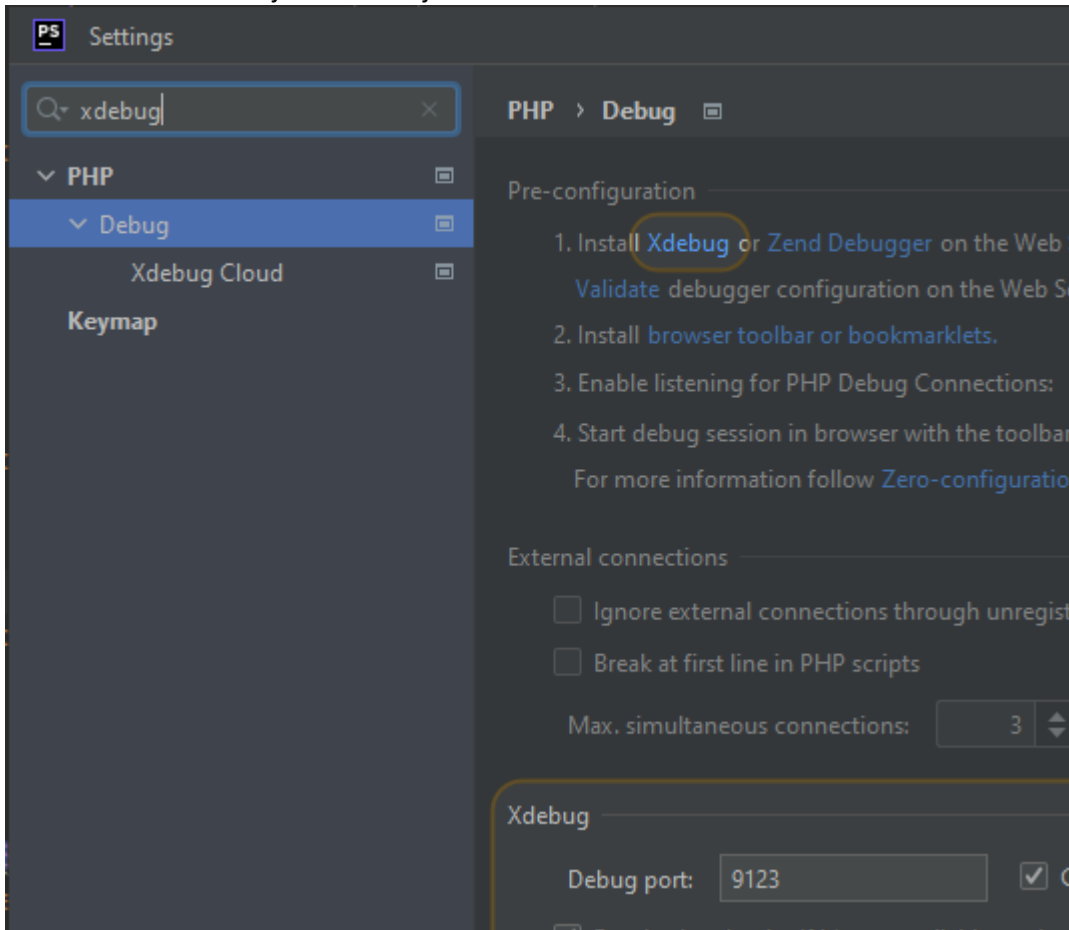
```
zend_extension = xdebug
xdebug.mode = debug
xdebug.start_with_request = yes
xdebug.client_port = 9123
```

Wyróżniony powyżej port 9123 należy zmienić na inny, losowy, unikalny. W stronie PHP INFO powinien pojawić się wpis o XDEBUG:

This program makes use of the Zend Scripting Language Engine:
 Zend Engine v4.2.11, Copyright (c) Zend Technologies
 with Xdebug v3.2.2, Copyright (c) 2002-2023, by Derick Rethans

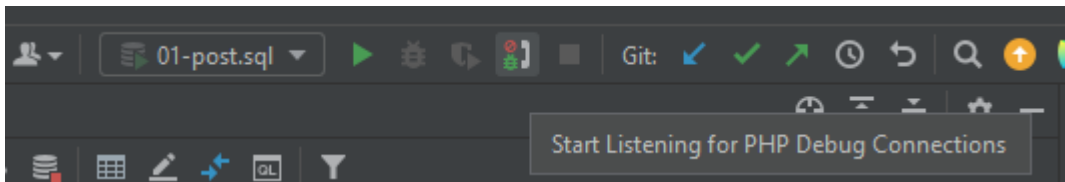
zendengine

W PhpStorm kliknij File -> Settings (Ctrl + Alt + S). Wyszukaj xdebug. Ustaw port zgodnie z powyżej wylosowanym:



Zatwierdź zmiany.

Otwórz plik public/index.php projektu i kliknij obok jednej z pierwszych instrukcji kodu, żeby utworzyć pułapkę (breakpoint). W górnym pasku PhpStorm kliknij ikonkę słuchawki (Start Listening (...)):

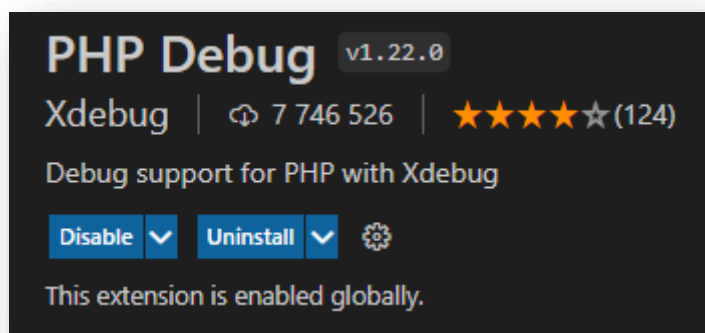


Odśwież stronę w przeglądarce. Wykonywanie powinno się zatrzymać, a PhpStorm powinien wyświetlać informacje o stanie zmiennych w pamięci i umożliwiać przejście przez aplikację krok po kroku.

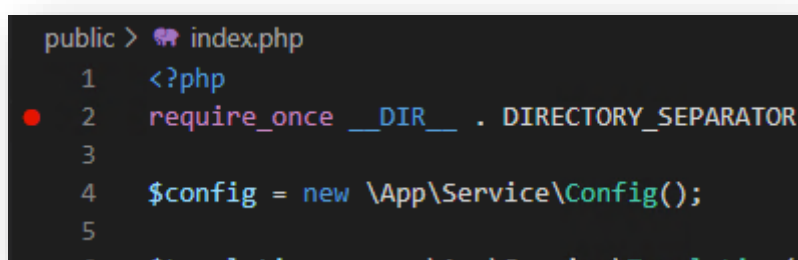
(ALTERNATYWNIE) VS CODE

Jeśli z jakiegoś powodu nie możesz lub nie chcesz korzystać z PhpStorm – spróbuj jeszcze raz. Jeśli dalej nie możesz, poniżej instrukcja na VS Code – ale przez resztę semestru będzie Ci o wiele trudniej!

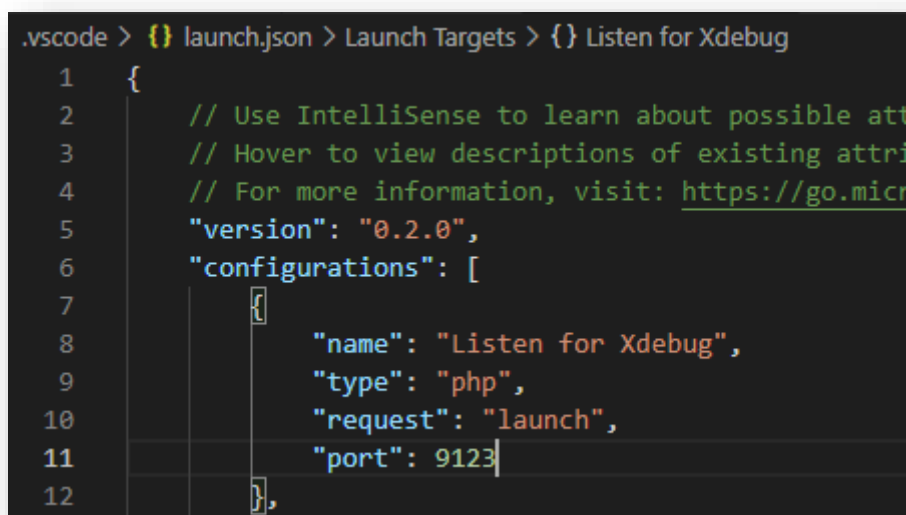
W Visual Studio Code zainstaluj rozszerzenie PHP Debug:



Otwórz plik index.php projektu i kliknij obok numeru linii przy pierwszych instrukcjach, utworzony zostanie breakpoint:



Kliknij menu Run -> Add Configuration... . Utworzony zostanie plik launch.json. Edytuj pozycję port, na port wskazany w konfiguracji XDEBUG w php.ini:

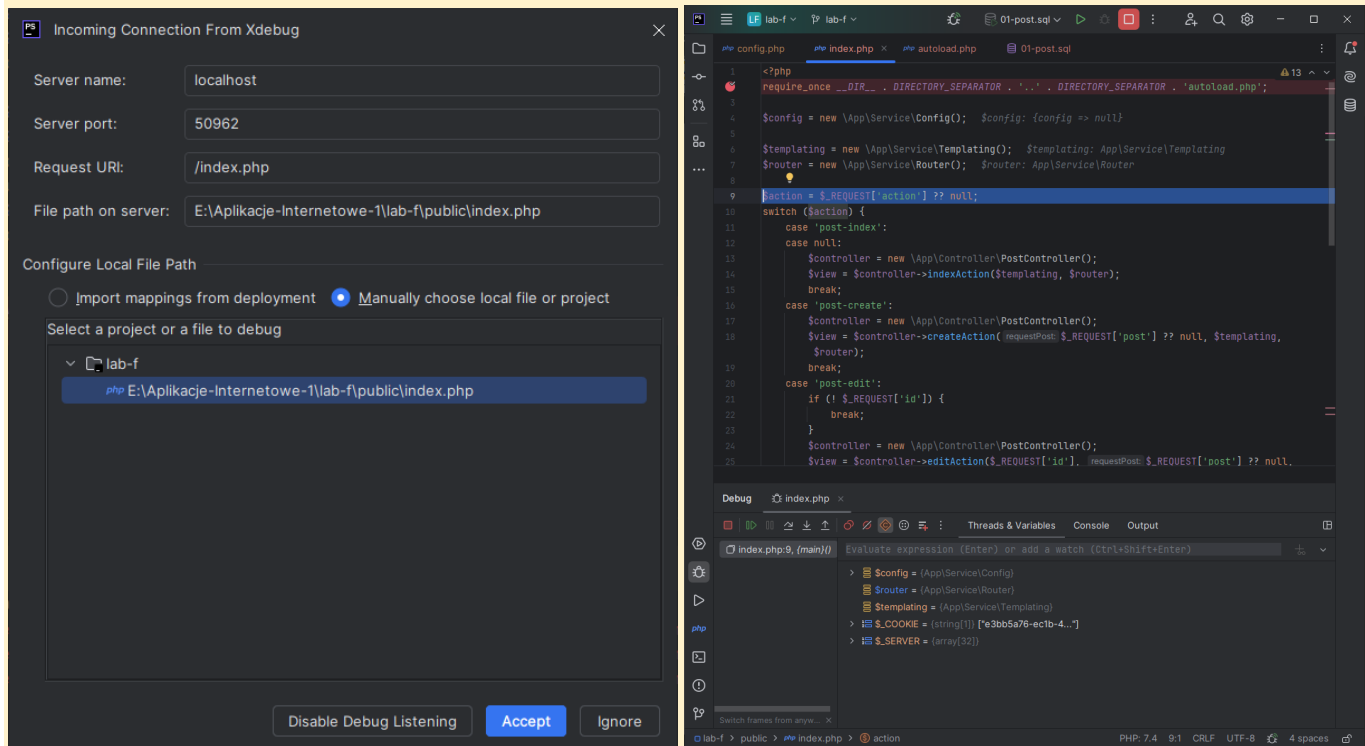


Kliknij menu Run -> Start Debugging (F5). Rozpocznie się oczekiwanie na połączenie:



ZATRZYMANIE W PUŁAPCE

Wstaw zrzut ekranu potwierdzający skuteczne zatrzymanie aplikacji na pułapce (breakpoint) w IDE (PhpStorm lub VSCode).



Punkty:

0

1

OMÓWIENIE FRAMEWORKA

Połącz się z prowadzącym na spotkaniu TEAMS. Za pomocą udostępniania ekranu, prowadzący omówi strukturę frameworka, obejmującą:

- plik konfiguracyjny config.php i config.dist.php
- organizację assetów – public/assets/src i public/assets/dist
- propozycja organizacji style.less i załączonych stylów oraz katalogu public/assets/src/vendor
- znaczenie i propozycja wykorzystania katalogu sql na migracje
- plik index.php – front-controller
- katalog Controller na przykładzie PostController – akcje związane z postami
- przykładowy model w Post.php
- serwisy w katalogu src/Service – czytnik konfiguracji, router, szablon
- szablony i sposób ich dziedziczenia w katalogu templates

Uważnie analizuj pokazywany kod i zadawaj pytania, aby zrozumieć.

Miejsce na Twoje notatki:

Głównie działające są: wyświetlanie ogólnej listy gier, dodawanie gier do tabeli.

Do naprawienia: usuwanie z tabeli, wypełnienie danych podczas edycji z poziomu listy głównej, wyświetlanie rekordu po kliknięciu Details, możliwość wejścia do edycji z poziomu pojedynczego rekordu tabeli.

Jedyny trop na ten moment, to znalezienie sposobu na poprawną zmianę linku podczas wybranych działań.

Przykładowo edycja z poziomu pojedynczego rekordu nie posiada informacji o id, mimo zawarcia w kodzie.

```
localhost:50962/index.php?action=game-edit
```

Jak być powinno:

```
localhost:50962/index.php?action=game-edit&gameId=1
```

...notatki...

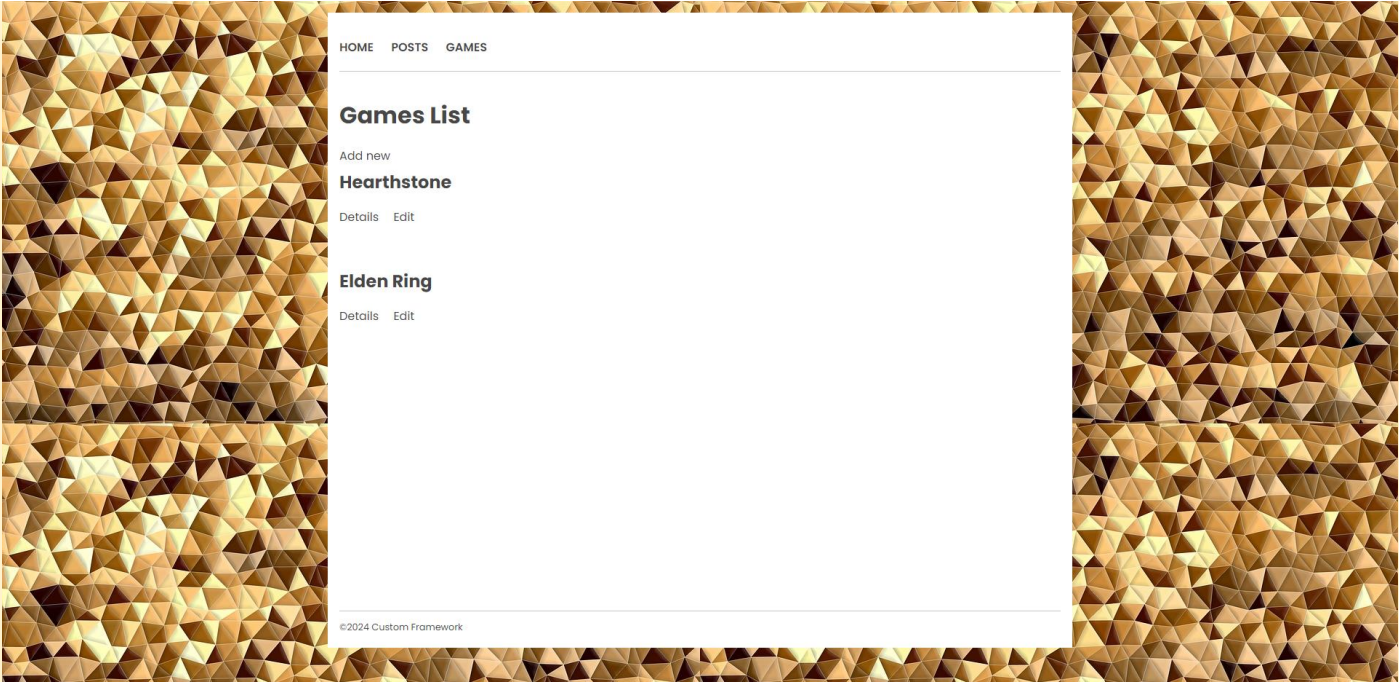
ROZSZERZENIE APLIKACJI

Wymyśl inny model podobny do prezentowanego `Post`. Rozszerz aplikację o implementację swojego modelu oraz związane z nim akcje – lista, prezentacja, tworzenie, edycja, kasowanie.

Wstaw zrzut ekranu kodu listy (indexAction):

```
public function indexAction(Templating $templating, Router $router): ?string
{
    $games = Game::findAll();
    $html = $templating->render( template: 'game/index.html.php', [
        'games' => $games,
        'router' => $router
    ]);
    return $html;
}
```

Wstaw zrzut ekranu listy (z danymi):



Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu kodu podglądu (showAction):

```
public function showAction(int $gameId, Templating $templating, Router $router): ?string
{
    $game = Game::find($gameId);
    if (! $game) {
        throw new NotFoundException( message: "Missing game with id $gameId");
    }

    $html = $templating->render( template: 'game/show.html.php', [
        'game' => $game,
        'router' => $router
    ]);
    return $html;
}
```

Wstaw zrzut ekranu podglądu:



Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu kodu tworzenia (createAction):

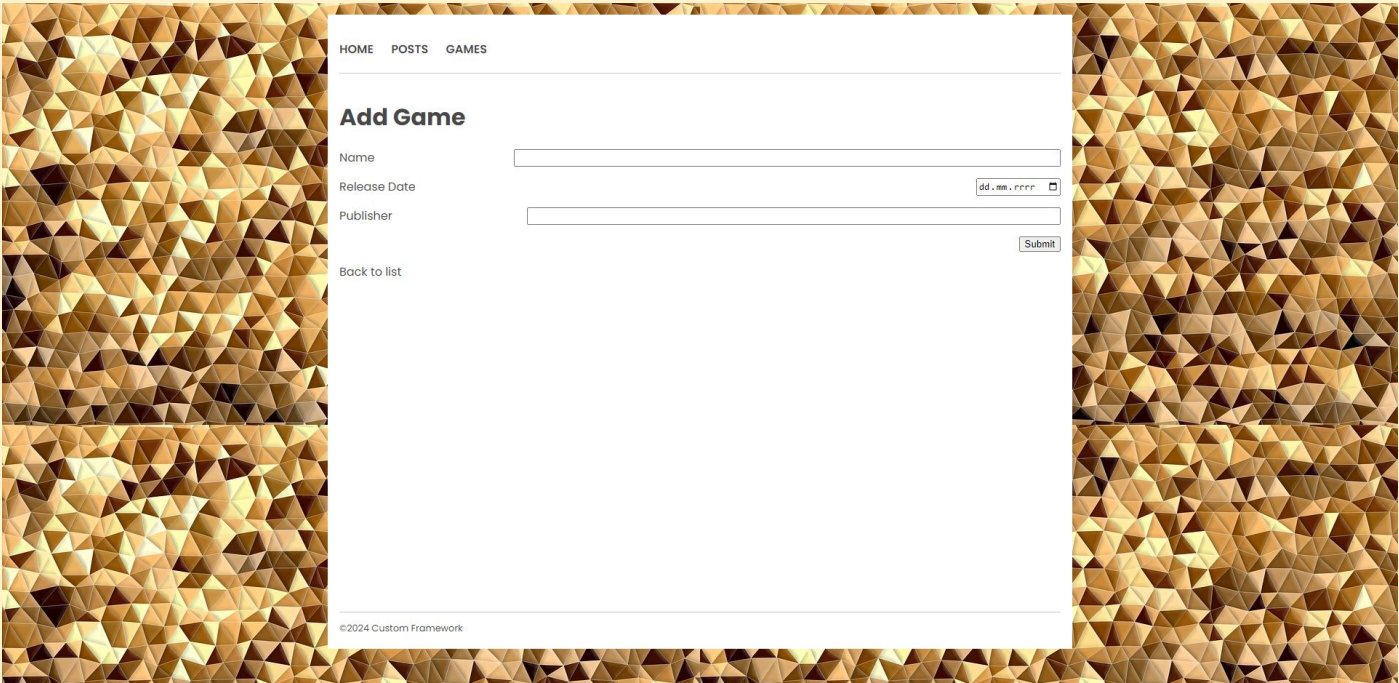
```
public function createAction(?array $requestGame, Templating $templating, Router $router): ?string
{
    if ($requestGame) {
        $game = Game::fromArray($requestGame);

        $game->save();

        $path = $router->generatePath( action: 'game-index');
        $router->redirect($path);
        return null;
    } else {
        $game = new Game();
    }

    $html = $templating->render( template: 'game/create.html.php', [
        'game' => $game,
        'router' => $router
    ]);
    return $html;
}
```

Wstaw zrzut ekranu tworzenia:



Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu kodu edycji (editAction):

```
public function editAction(int $gameId, ?array $requestGame, Templating $templating, Router $router): ?string
{
    $game = Game::find($gameId);
    if (! $game) {
        throw new NotFoundException( message: "Missing game with id $gameId");
    }

    if ($requestGame) {
        $game->fill($requestGame);

        $game->save();

        $path = $router->generatePath( action: 'game-index');
        $router->redirect($path);
        return null;
    }

    $html = $templating->render( template: 'game/edit.html.php', [
        'game' => $game,
        'router' => $router
    ]);
    return $html;
}
```

Wstaw zrzut ekranu edycji:



Punkty:

0

1

Wstaw zrzut ekranu kodu kasowania (deleteAction):

```

public function deleteAction(int $gameId, Router $router): ?string
{
    $game = Game::find($gameId);
    if (! $game) {
        throw new NotFoundException( message: "Missing game with id $gameId");
    }

    $game->delete();
    $path = $router->generatePath( action: 'game-index');
    $router->redirect($path);
    return null;
}

```

Punkty:	0	1
---------	---	---

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do **swojego** repozytorium GIT.

Upewnij się, czy wszystko dobrze się wysłało. Jeśli tak, to z poziomu przeglądarki utwórz branch o nazwie `lab-f` na podstawie głównej gałęzi kodu.

Podaj link do brancha `lab-f` w swoim repozytorium:

<https://github.com/Wojtuch566/Aplikacje-Internetowe-1/tree/lab-f>

...link, np. <https://github.com/inazwisko/ai1-lab/tree/lab-f...>

PODSUMOWANIE

W kilku zdaniach podsumuj zdobyte podczas tego laboratorium umiejętności.

...podsumowanie...

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.